

School of Computer Science and Artificial Intelligence

Lab Assignment # 13.2

Program	: B. Tech (CSE)
Specialization	:
Course Title	: AI Assisted coding
Course Code	:
Semester	: II
Academic Session	: 2025-2026
Name of Student	: M.Srinath
Enrollment No.	: 2403A51L29
Batch No.	: 51
Date	:10-03-2026

Task1:Use AI-assisted coding techniques to restructure a Python program that performs repeated calculations, transforming it into a reusable and maintainable design.

Prompt: Identify repeated computations in the following Python code. Refactor it by encapsulating repeated logic into a reusable function. Ensure output remains unchanged and add proper docstrings.

Code Screenshot:

```
def rectangle_metrics(length, width):  
    """  
    Calculate area and perimeter of a rectangle.  
  
    Parameters:  
        length (int or float): Length of rectangle  
        width (int or float): Width of rectangle  
  
    Returns:  
        tuple: (area, perimeter)  
    """  
    area = length * width # compute area  
    perimeter = 2 * (length + width) # compute perimeter  
    return area, perimeter  
  
# Test values  
rectangles = [(8, 4), (12, 6)]  
  
for length, width in rectangles:  
    area, perimeter = rectangle_metrics(length, width)  
    print("Area:", area)  
    print("Perimeter:", perimeter)
```

Code Output:

```
... Area: 32
    Perimeter: 24
    Area: 72
    Perimeter: 36
```

Code Explanation:

Original code repeated area/perimeter calculations twice.

Refactoring:

- Created reusable function `rectangle_metrics()`
- Used loop for multiple inputs
- Removed duplication → maintainable +

scalable Output remains unchanged.

Task 2: Use AI to refactor a Python script by extracting repeated inline calculations into reusable and well-documented functions.

Prompt: Find repeated inline calculations in this Python script and convert them into reusable, well-documented functions while improving readability.

Output Screenshot:

```
... Final Amount: 900.0
    Final Amount: 1800.0
```

Explanation:

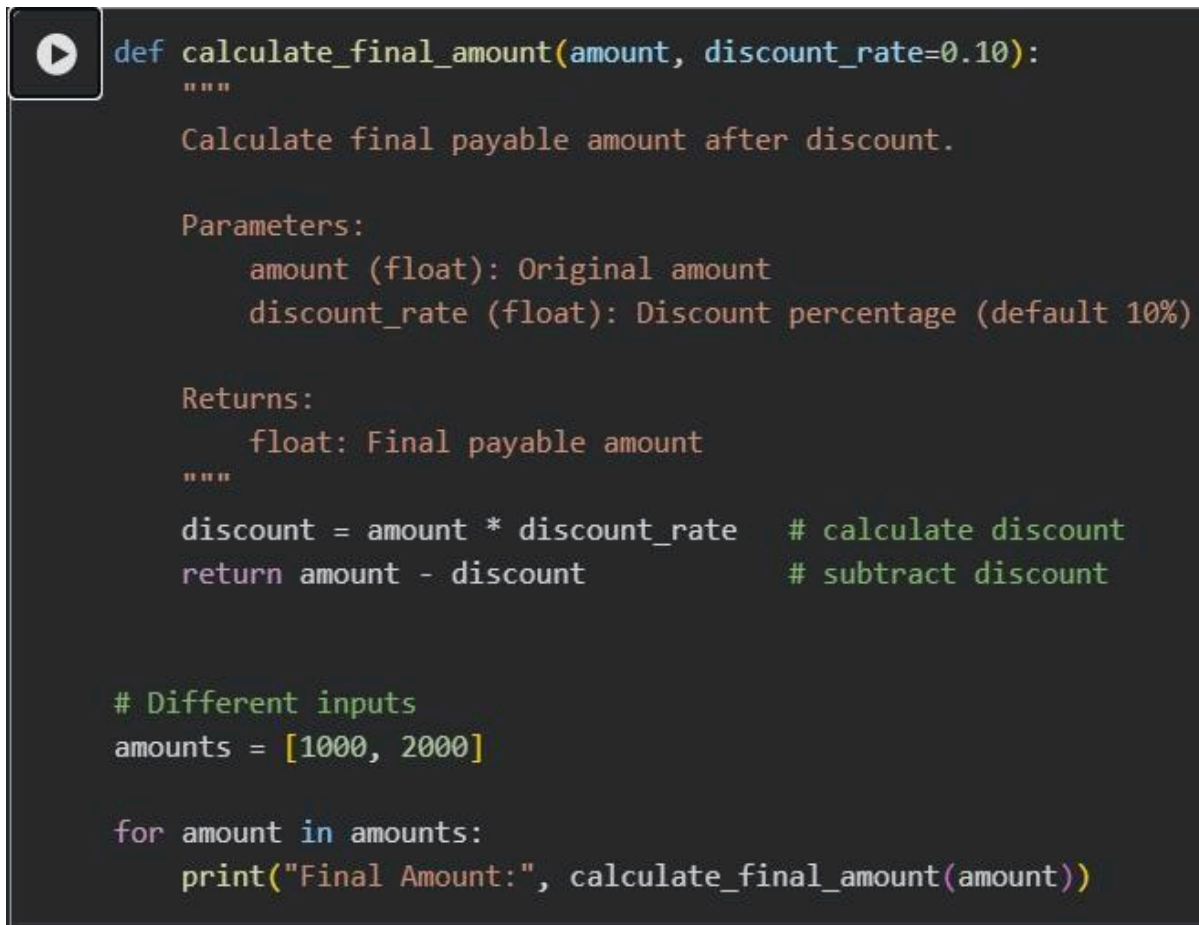
Original script repeated inline logic:

- discount calculation
- subtraction

logic Refactoring

benefits:

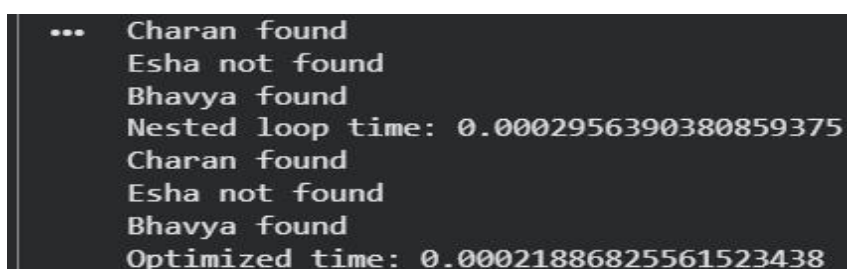
- Function reuse
- Easier rate change
- Cleaner code structure

Code Screenshot:A screenshot of a code editor with a dark background. It shows a Python function definition for calculating the final payable amount after a discount. The function has two parameters: 'amount' and 'discount_rate' (defaulting to 0.10). It includes docstrings for the function's purpose, parameters, and return value. Below the function definition, there are test inputs and a loop that prints the final amount for each input.

```
def calculate_final_amount(amount, discount_rate=0.10):  
    """  
    Calculate final payable amount after discount.  
  
    Parameters:  
        amount (float): Original amount  
        discount_rate (float): Discount percentage (default 10%)  
  
    Returns:  
        float: Final payable amount  
    """  
    discount = amount * discount_rate    # calculate discount  
    return amount - discount             # subtract discount  
  
# Different inputs  
amounts = [1000, 2000]  
  
for amount in amounts:  
    print("Final Amount:", calculate_final_amount(amount))
```

Task 3: Apply AI-based suggestions to enhance the performance of Python code that relies on nested loops and repeated conditional checks.

Prompt: Analyze the nested loop code and recommend more efficient logic or data structures. Refactor while preserving behavior and compare execution performance.

Output Screenshot:A screenshot of a terminal window showing the output of the code. It displays the results of the function calls for the inputs 1000 and 2000, and includes timing information for both the original nested loop code and the optimized version.

```
... Charan found  
    Esha not found  
    Bhavya found  
Nested loop time: 0.0002956390380859375  
Charan found  
Esha not found  
Bhavya found  
Optimized time: 0.00021886825561523438
```

Code Screenshot:

```
import time

students = ["Anil", "Bhavya", "Charan", "Divya"]
check_list = ["Charan", "Esha", "Bhavya"]

# ----- BEFORE (Nested Loop) -----
start = time.time()

for name in check_list:
    exists = False
    for student in students:
        if name == student:
            exists = True

    if exists:
        print(name, "found")
    else:
        print(name, "not found")

end = time.time()
print("Nested loop time:", end - start)

# ----- AFTER (Optimized using set) -----
start = time.time()

student_set = set(students) # faster lookup O(1)

for name in check_list:
    if name in student_set:
        print(name, "found")
    else:
        print(name, "not found")

end = time.time()
print("Optimized time:", end - start)
```

Explanation:

Original complexity:

- Nested loops $\rightarrow O(n \times m)$

m) Refactored:

- Used set $\rightarrow$ lookup $O(1)$
- Total $\rightarrow O(n + m)$

Task 4: Utilize AI assistance to improve maintainability by

replacing hardcoded numerical values with named constants.

Prompt: Detect hardcoded numeric values in this program and replace them with named constants to improve maintainability.

Code Screenshot:

```
# Constants
RATE = 2      # interest rate (%)
TIME = 5      # time in years

def simple_interest(principal):
    """
    Calculate simple interest.

    Parameters:
        principal (float): Principal amount

    Returns:
        float: Simple interest
    """
    return (principal * RATE * TIME) / 100

# Test values
principals = [1000, 2000]

for p in principals:
    print("Simple Interest:", simple_interest(p))
```

Code Output:

```
... Simple Interest: 100.0
    Simple Interest: 200.0
```

Explanation:

Original issue:

- Hardcoded values reduce flexibility

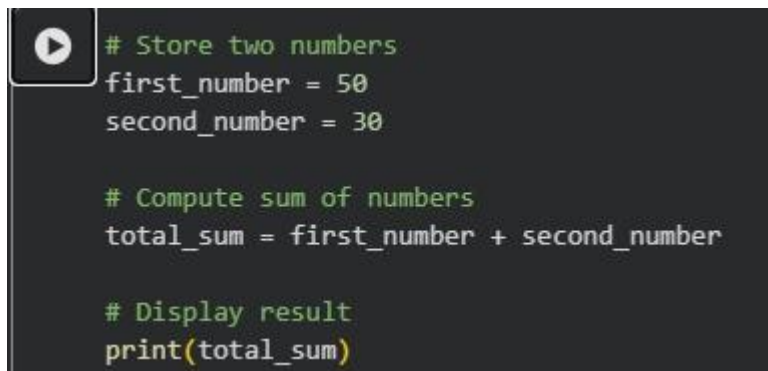
Refactoring:

- Introduced constants → readable
- Easier modification

Task 5: Utilize AI assistance to improve maintainability by replacing hardcoded numerical values with named constants.

Prompt: Improve readability by renaming unclear variables and adding meaningful comments while keeping program output unchanged.

Code Screenshot:

A screenshot of a code editor showing Python code. The code is color-coded: comments are green, variable names and numbers are white, and the print function is yellow. The code is as follows:

```
# Store two numbers
first_number = 50
second_number = 30

# Compute sum of numbers
total_sum = first_number + second_number

# Display result
print(total_sum)
```

Code Output:

A screenshot of a terminal window showing the output of the code. The output is the number 80, preceded by three dots.

```
... 80
```

Explanation:

Original:

- x, y, z

unclear

Refactored:

- Meaningful names
- Added comments

Improves readability + maintainability.

Conclusion:

- Refactoring improves software quality by reducing redundancy and enhancing maintainability.
- Using AI-assisted techniques helps identify repeated logic quickly and accurately.
- Encapsulating computations into reusable functions promotes modular design.
- Replacing hardcoded values with constants increases flexibility and readability.
- Optimizing loops and conditions improves execution efficiency and performance.
- Meaningful variable naming and documentation enhance code clarity.
- Refactored programs are easier to scale, debug, and maintain over time.
- Overall, AI-supported refactoring leads to cleaner, efficient, and professional code.